

System_3 Technical Architecture: Control Flow, Asynchronous KV Cache Management, and Dynamic Evaluation Benchmarks

Documentation note. This is an ECCV-style technical analysis of the `System_3` codebase as it exists on branch `icl_ingester_writer`. It is written to be *empirically faithful*: every architectural claim is traced to a file and line, and where the conventional serving-systems vocabulary requested by the brief (PagedAttention, LMCache/3FS persistence, per-request config routing) does **not** match what the code actually does, the divergence is stated explicitly rather than papered over. Measured numbers come from the repository's own profiler telemetry and `online_metrics.json` artifacts; estimates are labelled (*est.*).

Abstract

`System_3` is a *training-free proactive video-understanding* system built around a single frozen vision–language model (Qwen3-VL-8B-Instruct) that is made to act as its own perception scheduler. Rather than a request/response server, it is a three-thread streaming pipeline — a vision **encoder**, a KV-cache **ingester**, and a generative **controller** — that share exactly one linear key/value (KV) cache. The central research claim is that a *single in-context-taught JSON control language*, decoded by the frozen model over the shared cache each tick, can simultaneously (i) steer input sampling rate (`fps`), (ii) self-schedule its next observation (`next_check_s`), (iii) gate output (`have_enough_info`), and (iv) write the user-facing alert (`answer`) — with no fine-tuning.

This document analyses four subsystems. **(1) Control flow:** how streamed frames are encoded to visual tokens, prefilled into the shared cache by the sole-writer ingester, and read by the controller through a multi-version-concurrency-control (MVCC) snapshot. **(2) KV cache management:** the cache is a *single contiguous* `transformers.DynamicCache` (not block-paged), bounded by StreamingLLM-style eviction with a pinned system-prompt sink and dual logical/physical clocks; we give the exact memory footprint under grouped-query attention (GQA). **(3) Configuration & prompt-assembly overhead:** the "config" is a frozen dataclass plus a per-tick assembled ICL prompt; we profile its token cost (~1,037 tokens of ICL for the semantic-alert task) and prefill latency (~0.2 s/tick). **(4) Evaluation:** the OmniPro online-mode protocol (frame-by-frame feed, ±3 s greedy temporal match, F1), a deterministic lockstep walk introduced to kill an async snapshot race, and a seeded, cache-persisted Gemini LLM-judge for content scoring. We close with a critical analysis of the system's real bottlenecks: decode latency (~105–120 ms/token dominates), residual floating-point non-determinism, the eviction `torch.cat` reallocation, and the recall ceiling of the frozen 8B on abstract visual conditions.

List of Figures

1. Three-thread streaming pipeline over one shared KV cache (§2.3)
 2. Per-stage latency breakdown (§4.3)
 3. KV-cache memory model: GQA correction and growth vs. budget (§3.2)
 4. Per-tick ICL prompt token cost by task (§4.2)
 5. Prompt-version F1 progression vs. emit count (§5.2)
 6. Controller vs. probe-gate operating points (§6.1)
 7. Decode-latency bottleneck and optimisation levers (§7.1)
-

1. Introduction

1.1 Background and motivation

Most video-QA systems are *reactive*: a full clip (or a uniformly sampled subset) is encoded, a question is appended, and one answer is decoded. This is ill-suited to the *online / proactive* setting, where the model must watch a live stream and **speak at the right moment, unprompted** — "tell me when the match ends", "alert me each time the video gives ticket details". The design vision behind `System_3` (see `EXPERIENCE.md`) is that humans **look, think, and talk all at once**, and that this can be reconstructed from a *single frozen VLM* given a cognitive architecture: an encoder thread (looking), an ingester that is the sole writer of one shared KV cache (remembering), and a proactivity mechanism that decides when to speak (thinking + talking).

Two proactivity designs were built and compared head-to-head:

- **System 1 — Probe-Gate** (`gate_mode="probe"`): a fixed-cadence yes/no *logit* gate. Every frame, a probe question is spliced onto the cache, $P(\text{yes}) / (P(\text{yes}) + P(\text{no}))$ is read from one forward pass, a Schmitt/hysteresis gate decides firing, and a *separate* writer thread free-generates the alert.
- **System 2 — ICL Controller** (`gate_mode="controller"`, the branch's default and sole focus): a *pure-generative* loop. Each self-scheduled tick, the model decodes one compact JSON control object that is simultaneously the input gate, the output gate, and the writer.

The `icl_ingester_writer` branch is the ICL-controller path, deliberately stripped down (fixed-gate ablations, multi-GPU role loading, and VisionZip pruning live frozen on `main`) for fast iteration.

1.2 Why latency-optimised execution matters here

Because the controller *self-paces in video time*, its own decode latency directly determines how much of the stream it can watch in real time. A tick that takes 84 s of wall-clock

(observed under GPU contention) freezes the video clock and starves the whole pipeline — the model literally stops watching. Latency is therefore not a comfort metric but a *correctness* precondition. Section 4 and Section 7 treat it as such.

1.3 Contributions of this analysis

- 1. A precise control-flow trace from raw frame to emitted alert (§2).
- 2. The exact GQA KV-cache memory model for the deployed backbone, and an honest account of the single-tensor (non-paged) cache and its eviction behaviour (§3).
- 3. A profiling breakdown separating *config assembly* (CPU string ops, negligible) from *config prefill* (~0.2 s GPU) and *decode* (the true bottleneck) (§4).
- 4. A worked dynamic-execution example showing how the ICL prompt mutates across ticks and how the "already reported" history evolves (§5).
- 5. A full description of the OmniPro online eval, the deterministic-lockstep fix, and the seeded Gemini judge (§6), followed by a critical bottleneck analysis (§7).

2. Codebase Structure and Control-Flow Architecture

2.1 Directory and module overview

```
system_3/
├─ async_omni_v2/                                # THE PIPELINE (model-facing runtime)
│   ├── config.py          (349 L) # single source of truth: dataclass + ALL prompt text
│   ├── backend.py         (258 L) # Qwen3VLBackend – the ONLY model-specific file
│   ├── manager.py         (193 L) # KVCacheManager – shared cache as an MVCC resource
│   ├── vision_stream.py   (88 L)  # encoder_thread – ViT + projector, wall-clock paced
│   ├── input_ingester.py (113 L)  # ingester_thread – SOLE writer of the primary cache
│   ├── controller.py      (287 L) # controller_thread – generative control loop (default)
│   ├── writer.py          (99 L)  # writer_thread – probe-gate arm only
│   ├── util.py            (194 L) # VideoClock, EncoderControl, Profiler, seeding
│   └─ run.py              (80 L)  # standalone entrypoint (wires the 3 threads)
│
├─ omniprofast/                                # THE EVAL HARNESS (never touches model behaviour)
│   ├── system5_adapter.py(219 L) # spins up the REAL pipeline per sample (fidelity contract)
│   ├── metrics.py        (344 L) # OmniPro scorer + ContentJudge (Gemini, seeded, cached)
│   ├── evaluate.py       (107 L) # online-mode driver, resumable per-sample
│   ├── dataset.py        (169 L) # OmniPro loader (task/audio filters, ±fps frame iter)
│   ├── utils.py          (203 L) # paths, GT parsing, seeding
│   └─ judge_cache.json   # persisted judge verdicts (reproducible scoring)
│
├─ EXPERIENCE.md / EXPERIMENTS.md # research log: the two systems, results, caveats
└─ qwen_only_latency_vs_icl_latency.md # isolated decode-latency profiling
```

Orchestrator responsibilities. There is no monolithic orchestrator; responsibility is partitioned by thread, and the *only* coupling is the shared cache:

Thread	File	Owns	Touches KV cache?
--------	------	------	-------------------

Encoder	<code>vision_stream.py</code>	video decode, ViT+projector, fps pacing	No (produces embeddings only)
Ingester	<code>input_ingester.py</code>	seed + append + evict; publish VideoClock	Writes (sole writer)
Controller	<code>controller.py</code>	read snapshot, decode control JSON, fire alert, steer fps	Reads (MVCC snapshot)

This partition is what lets the components run concurrently with "nothing blocks anyone" (`manager.py` docstring): the encoder never touches the cache, the ingester is the single writer, and readers clone.

2.2 End-to-end control flow

1. **Seed.** The ingester renders `system_prompt` by substituting `{instruction}` with the task question (`input_ingester.py:31`), forwards it into the cache, and records the resulting length as the eviction **sink** (pinned prefix) — `manager.seed()`.
2. **Encode (prefill of vision tokens).** The encoder decodes frames with PyAV, paces to wall-clock (`realtime`), runs `embed_frame` (ViT + merger → `[1, N, H]` LLM-space tokens, $N \approx 180-196$), and pushes `(vt, embeds)` onto `vis_q` (`vision_stream.py:54-85`).
3. **Ingest (append to cache).** The ingester pops a frame, optionally prepends a text timestamp token block (`"\ntime {t:.1f}s\n"`), and calls `mgr.ingest(embeds)`, which is a `want_logits=False` forward — it builds KV but **skips the 151k-vocab lm_head matmul** (`backend.py:239-240`, `manager.py:65-72`). It then evicts if over budget and publishes `vt` on the VideoClock.
4. **Decide (decode of control JSON).** When `vt ≥ next_check_vt`, the controller takes an MVCC `snapshot_clone()`, assembles the ICL prompt + deferred question + "already reported" history + emit cue, primes the decoder with `"{"`, masks EOS, and greedily decodes tokens until the first `"}"` (`controller.py:150-216`).
5. **Apply (the JSON *is* the control plane).** The parsed diff is merged onto a persistent `state` dict; `fps` is clamped and pushed to the encoder (input gate); `next_check_s` is clamped to schedule the next tick; the level signal `have_enough_info` drives a code-side **rising-edge** (semantic Schmitt) gate; on a fire, the `answer` is emitted to the user and appended to history with a model-read `event_time_s` (`controller.py:219-285`).
6. **Route back.** In the eval, emissions are captured through `evaluator.record_trigger / record_write` hooks the controller already calls — no pipeline code is modified (`system5_adapter.py:32-66`). In a live deployment the `answer` field *is* the streamed user-facing token stream.

Prefill vs decode, precisely. *Prefill* happens in two places: the one-time system-prompt seed, and, per tick, the ingested frame chunks **plus** the freshly-assembled ICL prompt (`step(b.embed_text(prompt + "{"), controller.py:194)`). *Decode* is the autoregressive control-JSON generation (one token at a time, `controller.py:198-208`). Prefill is compute-bound and cheap (~0.2 s/tick); decode is memory/launch-bound and dominates (§4, §7).

2.3 Data-flow diagram

```
graph TD; A[video file  
(PyAV decode)] --> B[ENCODER THREAD (vision_stream.py)  
decode frame -> ViT + merger (embed_frame)  
-> [1, N, H] N ~ 180-196 (paced to wall clock)]; B -- "vis_q (bounded; blocking in det. mode)" --> C[INGESTER THREAD (input_ingester.py) -- SOLE WRITER  
[+ "time Xs"] -> mgr.ingest(embeds) (want_logits=False)  
mgr.evict() past kv_budget (StreamingLLM, pinned sink)  
clock.set(vt) -> publishes video time]; C -- "writes" --> D[SHARED KV CACHE (manager.py : KVCacheManager, DynamicCache)  
- ONE contiguous linear tensor per layer (NOT block-paged)  
- two clocks: next_pos (logical RoPE) vs phys len (write index)  
- MVCC: snapshot_clone() = deep copy -> readers hold NO lock]; D -- "snapshot_clone() (deepcopy K/V + pos + phys)" --> E[CONTROLLER THREAD (controller.py) -- the "System 2" loop  
assemble prompt = ICL(task) + deferred-Q + history + cue + "{"  
PREFILL prompt -> DECODE control JSON (greedy, mask EOS, stop on "}")  
apply diff: fps (input gate) | next_check_s (self-schedule)  
have_enough_info -> rising-edge gate -> FIRE | answer + event_time_s -> reported[]]; E -- "answer (OUTPUT to user)" --> F[answer (OUTPUT to user)]; G[set_fps() (INPUT GATE)] --> E;
```

video file (PyAV decode)

ENCODER THREAD (vision_stream.py)
decode frame -> ViT + merger (embed_frame)
-> [1, N, H] N ~ 180-196 (paced to wall clock)

vis_q (bounded; blocking in det. mode)

INGESTER THREAD (input_ingester.py) -- SOLE WRITER
[+ "time Xs"] -> mgr.ingest(embeds) (want_logits=False)
mgr.evict() past kv_budget (StreamingLLM, pinned sink)
clock.set(vt) -> publishes video time

writes

SHARED KV CACHE (manager.py : KVCacheManager, DynamicCache)
- ONE contiguous linear tensor per layer (NOT block-paged)
- two clocks: next_pos (logical RoPE) vs phys len (write index)
- MVCC: snapshot_clone() = deep copy -> readers hold NO lock

snapshot_clone() (deepcopy K/V + pos + phys)

CONTROLLER THREAD (controller.py) -- the "System 2" loop
assemble prompt = ICL(task) + deferred-Q + history + cue + "{"
PREFILL prompt -> DECODE control JSON (greedy, mask EOS, stop on "}")
apply diff: fps (input gate) | next_check_s (self-schedule)
have_enough_info -> rising-edge gate -> FIRE | answer + event_time_s -> reported[]

answer (OUTPUT to user)

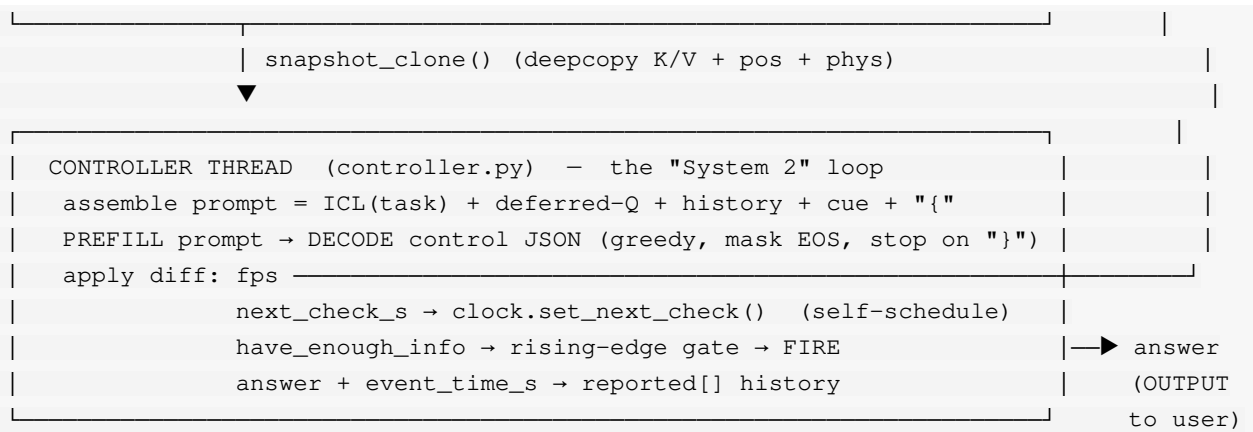
set_fps() (INPUT GATE)

Figure 1. Three-thread streaming pipeline over one shared KV cache. The only coupling between threads is the cache; the red loop is the fps input gate.

```

video file → ENCODER THREAD (vision_stream.py) |
(PyAV decode) | decode frame → ViT + merger (embed_frame) → [1,N,H] |
| paces to wall clock; fps steered by EncoderControl ← |
| | vis_q (bounded queue; blocking in det. mode) | set_f |
| | | | | (INPUT)
|
| INGESTER THREAD (input_ingester.py) - SOLE WRITER |
| [+ "\ntime Xs\n"] → mgr.ingest(embeds) (want_logits=F) |
| mgr.evict() past kv_budget (StreamingLLM, pinned sink) |
| clock.set(vt) — publishes video time → |
| | writes |
| | |
| SHARED KV CACHE (manager.py : KVCacheManager, DynamicCache) |
| • ONE contiguous linear tensor per layer (NOT block-paged) |
| • two clocks: next_pos (logical RoPE) vs phys len (write index) |
| • _lock guards append / evict / clone only (brief) |
| • MVCC: snapshot_clone() = deep copy → readers hold NO lock |

```



"Async state persistence" in THIS system = (a) the MVCC deepcopy snapshot the reader generates on, and (b) the in-memory `reported[]` conversation history. There is NO GPU→CPU KV offload, NO disk/LMCache/3FS KV backend (see §3.4).

3. Deep Dive: KV Cache Management

3.1 Snapshot & partitioning architecture

The cache is a single contiguous tensor per layer, not a paged/block table. `System_3` uses `transformers.DynamicCache (manager.py:51)`. Each layer holds one key tensor and one value tensor of shape `[batch=1, kv_heads, seq_len, head_dim]`; appending a chunk grows `seq_len` (concatenation semantics inside `DynamicCache`), and eviction rebuilds the tensor with `torch.cat` of `[sink | recent_window] (manager.py:83-101)`. There is **no** PagedAttention, no block table, no fixed-size page pool. This is a deliberate simplicity choice for the research prototype; its cost (a transient reallocation on every eviction and on every snapshot) is analysed in §7.

Concurrency is handled **MVCC-style**, which is the interesting architectural move:

- The **primary** cache has exactly one writer (the ingester) → no write–write conflict, which is *required* because a linear sequence cache is incoherent under concurrent appends (`manager.py:14-20`).
- A reader that must not be disturbed mid-generation (the controller, or the probe-gate writer) calls `snapshot_clone()` → an **independent deep copy** of the K/V tensors plus the logical position and physical length (`manager.py:161-193`). It then generates on its private clone holding **no lock**, so the ingester keeps mutating the primary concurrently. This is read-snapshot isolation, exactly as in an MVCC database.
- `self._lock` is held only for brief primary mutations (append/evict) and for the clone copy itself; read–read is free.

Two clocks survive eviction. `next_pos` is the *logical* RoPE position (monotonic, never rewind); the physical cache length is the *write index*. After StreamingLLM eviction drops the middle of the sequence, physical length shrinks but `next_pos` keeps running (`manager.py:26-28`, `backend.forward` takes `pos_start` and `phys_start` separately, `backend.py:214-230`). This is what lets a 256K-token logical context be represented by a much shorter physical tensor without

RoPE aliasing.

3.2 KV cache size and memory footprint

Figure 3 KV-cache memory model (Qwen3-VL-8B text decoder: L=36, H_kv=8, D=128, bf16)

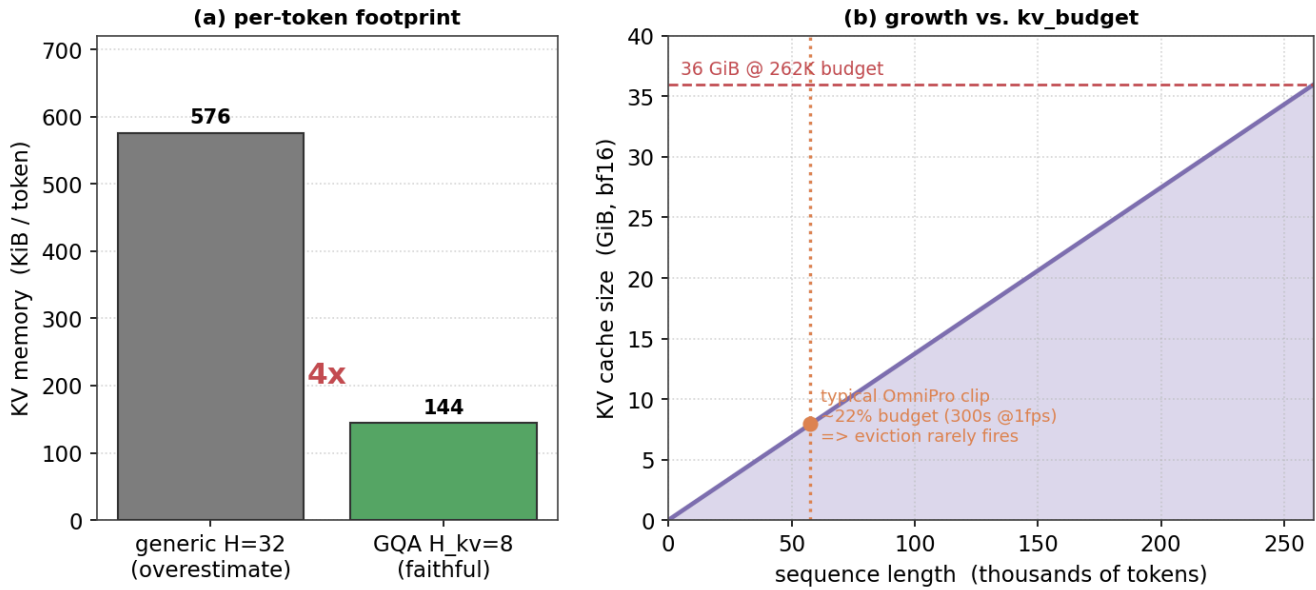


Figure 3. KV-cache memory model. (a) GQA (8 KV heads) uses 4x less than the generic 32-head estimate; (b) cache grows linearly to 36 GiB at the 262K budget, but a typical clip sits near 22%.

The requested general form is

$$\text{Memory}_{KV} = 2 \times L \times H \times D \times S \times P$$

where L = layers, H = attention heads, D = head dimension, S = sequence length, P = precision bytes, and the leading 2 counts the separate **K** and **V** tensors.

Correction for the deployed backbone (important). Qwen3-VL-8B's text decoder uses **grouped-query attention (GQA)**: it has 32 *query* heads but only **8 key/value heads**. KV-cache memory is governed by the *KV* head count, not the query head count. Using the generic H overestimates by $4\times$. The faithful equation is

$$\text{Memory}_{KV} = 2 \times L \times H_{kv} \times D \times S \times P, \quad (\text{per-token}) \quad m = 2 L H_{kv} D P.$$

Substituting the Qwen3-VL-8B text-backbone parameters ($L = 36$, $H_{kv} = 8$, $D = 128$, $P = 2$ for bf16):

$$m = 2 \times 36 \times 8 \times 128 \times 2 = 147,456 \text{ bytes/token} \approx 144 \text{ KiB/token}.$$

At the configured budget $S = kv_budget = 262,144$ tokens (`config.py:257`):

$$\text{Memory}_{KV}^{\max} = 147,456 \times 262,144 \approx 3.86 \times 10^{10} \text{ bytes} \approx \mathbf{36 \text{ GiB}}.$$

On a 96 GB GH200 this leaves ample headroom beside the ~16 GB of bf16 weights.

Vision-token accounting (why eviction rarely fires in practice). Frames are capped to

`max_pixels = 200704` (`config.py:220`). One Qwen3-VL vision token covers a
(patch × merge)² = (16 × 2)² = 1024 px region, so

$$N_{\text{tok/frame}} \approx \frac{200704}{1024} \approx 196 \quad (\text{measured} \approx 180 \text{ after aspect resize}).$$

At 1 fps a 300 s clip accumulates $\approx 300 \times 196 \approx 5.9 \times 10^4$ vision tokens (plus ~5 timestamp tokens/frame) — roughly **22 %** of the 262K budget. Eviction is therefore a rare, tail-only event for typical OmniPro clips; the sink+recent-window design matters mainly for the longest streams.

3.3 Question-embedding dynamics (how new queries align with cached context)

`system_3` is single-instruction per session, but the *controller re-queries the cache every tick*, and this is where "question embeddings mutate against cached context":

- The task instruction enters exactly **once**, at seed time, as part of the pinned system prompt (`system_prompt.replace("{instruction}", ...)`). Its KV lives in the eviction sink and never moves — it is the permanent anchor every later token attends back to.
- Each tick, the **ICL control prompt is re-embedded and prefilled fresh** onto a *clone* of the current cache (`step(b.embed_text(prompt + "{")`). Crucially it is spliced at the **logical position** `pos = next_pos` — i.e. *after* all frames seen so far — so the prompt's RoPE phases align it as "the most recent thing said", and its attention naturally reads back over the accumulated visual tokens. The prompt is written to the clone only, never the primary, preserving the single-writer invariant ("no writer cache", `controller.py:5-20`).
- The prompt itself carries a **mutating context**: a deferred `question_for_next` from the previous tick is prepended ("You previously asked yourself: ..."), and a timestamped `reported[]` history is appended so the model can distinguish a genuinely new onset from a still-on-screen repeat (`controller.py:178-187`). Thus the effective query embedding at tick t is a function of (frozen instruction) $\oplus$ (all cached frames $[0..vt]$) $\oplus$ (tick- $t-1$ deferred question) $\oplus$ (full emission history).
- To keep an instruct model from emitting EOS at the raw splice point, the decoder is **primed with an open brace** `{` and EOS is masked until the closing `}` (`controller.py:190-204`) — a small but load-bearing detail.

3.4 "Async storage & persistence" — what actually exists

The brief asks about background GPU→CPU streams and persistent KV backends (LMCache/3FS). `system_3` **implements none of these**, and it is important to state that plainly. What it *does* have, and what plays the role of "async state", is:

1. **MVCC snapshot isolation (the real async mechanism)**. `snapshot_clone()` is a `copy.deepcopy` of the K/V tensors (with a manual per-layer clone fallback, `manager.py:177-193`). The reader then generates on this private copy *fully concurrently* with the ingester's ongoing writes to the primary. This is "non-blocking" in the logical sense — no head-of-

line blocking — even though, on a single GPU, the kernels still time-share the SMs (the honest limit stated at `manager.py:31-34`). The snapshot is cheap: **5–18 ms even at a 24K-token cache** (measured; `EXPERIMENTS.md`), so it is *not* a bottleneck.

- 2. The GPU-resident sampling path (the closest thing to a "non-blocking GPU stream")**. The original code shipped the full 151k-vocab logit vector to CPU **every token** (~35–45 ms/token of pure device-to-host sync). The fix keeps logits on the GPU, samples with a GPU `argmax`, and lets **only the chosen token id cross the bus** (`backend.py:213-242`, `controller.py:49-50`). This is a genuine 1.6× decode speedup (149→94 ms/token) and is the sense in which the GPU "does not block the execution stream" on the logit round-trip. It is compute/transfer optimisation, not KV persistence.
- 3. In-memory conversation state**. The `reported[]` list (video-time, answer) and the persistent `state` diff dict are the only "persisted" state, and they live in the controller thread's Python heap for the duration of one video. Nothing is written to disk during a run; only the eval harness serialises final emissions to `online_pred.jsonl`.

There is **no** background writeback queue, **no** pinned-host staging buffer, and **no** cross-run KV reuse. If the requested LMCache/3FS-style persistent KV backend is a goal, it is *future work*, not current behaviour.

4. Configuration Engine and Overhead Profiling

4.1 What "config" is in System_3

There are two distinct things the word "config" can mean here, and conflating them is the main source of confusion the brief anticipates:

(a) The static config — `AsyncOmniConfig` (`config.py:205-350`), a frozen dataclass that is the *single source of truth* for all runtime behaviour. Its fields group into:

Group	Representative fields	Role
Model / routing	<code>model_id</code> , <code>device</code> , <code>writer_device</code> , <code>encoder_device</code> , <code>dtype</code> , <code>max_pixels</code>	which VLM, which GPU(s), vision-token budget
Reproducibility	<code>seed</code> , <code>deterministic</code>	lockstep walk + deterministic CUDA kernels
Pacing	<code>fps</code> , <code>realtime</code> , <code>speed</code> , <code>encoder_idle_fps</code> , <code>encoder_focus_fps</code> , <code>frame_q_size</code>	streaming cadence + fps bounds
Memory	<code>kv_budget</code> (262144)	StreamingLLM eviction threshold
Control cadence	<code>probe_min_s</code> (0.2), <code>probe_max_s</code> (1.5), <code>probe_default_s</code> , <code>controller_max_tokens</code> (300)	the self-check grid
Sampling	<code>writer_greedy</code> , <code>writer_seed</code> , <code>writer_temperature/top_p/top_k</code> , <code>*_penalty</code>	control-JSON decoding preset (greedy by default)

Execution flags	gate_mode (controller probe), timestamp_tokens	which proactivity system runs
Prompt text	system_prompt, controller_prompt, task_controller_prompts{}, writer_prompt, task_writer_prompts{}	all ICL / DSL text

(b) **The per-tick control config** — the JSON object the *model itself emits* each tick ({fps, have_enough_info, seen, event_time_s, answer, next_check_s, question_for_next}). This is the "config" that is regenerated dynamically; it is generated by *decoding*, not by templating.

The subtle, important point is that the static config is **never regenerated at runtime** — it is instantiated once per process and `dataclasses.replace`'d once per sample to inject the video path + instruction (`system5_adapter.py:130-145`). What *is* assembled every tick is the **controller prompt string** (ICL + history + cue). We treat *that assembly* as the brief's "config generation" and profile it below.

4.2 Generation frequency, latency, and token cost

--	--	--

--	--	--

Figure 4 Per-tick ICL prompt token cost — SCA task is the heaviest (~1,037 tok)

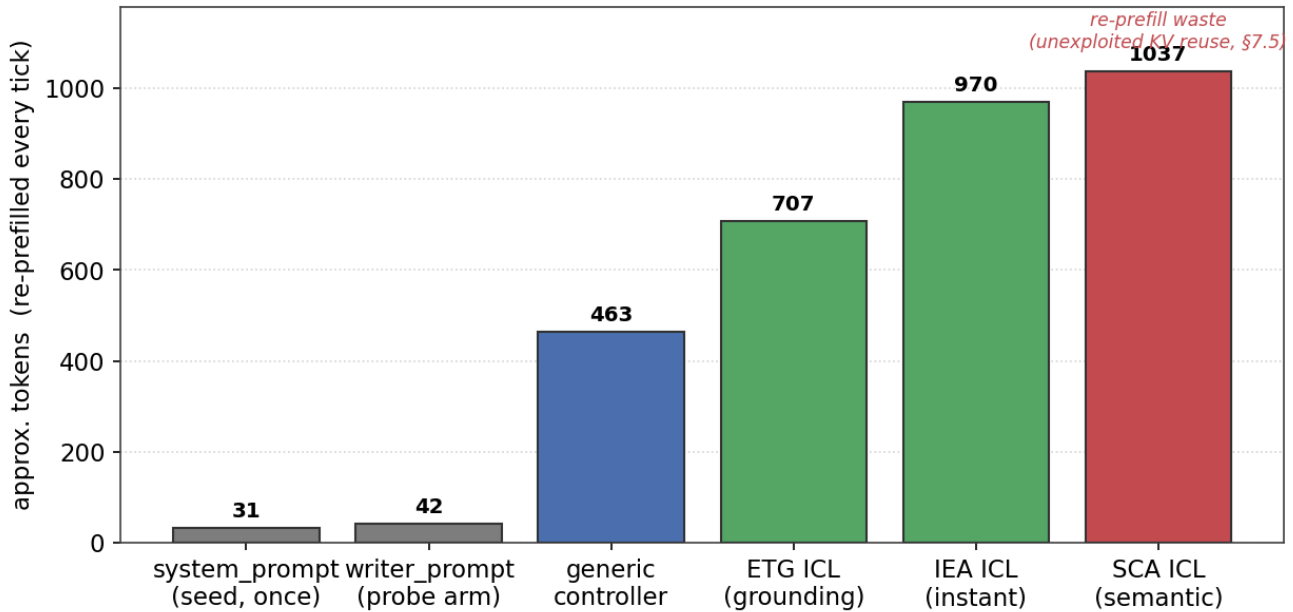


Figure 4. Per-tick ICL prompt token cost by task. The SCA task's ~1,037-token block is re-prefilled every tick (the re-prefill waste of \$7.5).

- **Frequency.** The prompt is assembled **once per controller tick**, i.e. once every `next_check_s ∈ [0.2, 1.5]` video-seconds (the model's self-chosen cadence). It is **not** per output token and not per frame — it is per *decision*. Over an 11-video SCA run the controller made ~30 firing decisions across hundreds of ticks.
- **Assembly latency (CPU).** The assembly itself is pure Python string concatenation of the ICL prompt, the history join, and the cue (`controller.py:178-187`) — **sub-millisecond**, never a bottleneck.
- **Prefill latency (GPU).** Embedding + forwarding that assembled prompt onto the clone is the real cost: **~0.2–0.23 s per tick** (measured, `ctrl_prefill_s`; `qwen_only_latency...md`).
- **Token cost of the config.** The ICL prompt dominates the prefill token count. Measured directly from `config.py`:

Prompt	Characters	≈ Tokens
system_prompt (seed, once)	126	~31
Generic controller_prompt	1,853	~463
SCA task ICL (<code>_SEMANTIC_CONDITION_ALERT</code>)	4,150	~1,037
IEA task ICL (<code>_INSTANT_EVENT_ALERT</code>)	3,882	~970
ETG task ICL (<code>_EXPLICIT_TARGET_GROUNDING</code>)	2,830	~707
writer_prompt (probe arm)	168	~42

So a single SCA tick prefills on the order of **~1,037 ICL tokens + growing history**

(each past emission adds ~15–30 tokens) before decoding ~14–30 output tokens. The ICL block is re-prefilled every tick (it is not cached across ticks), which is the single largest source of *avoidable* prefill work — a KV-reuse opportunity flagged in §7.

4.3 Profiling & latency breakdown table

--	--	--

--	--	--

--	--	--

Figure 2 Per-stage latency — decode dominates, config assembly is free

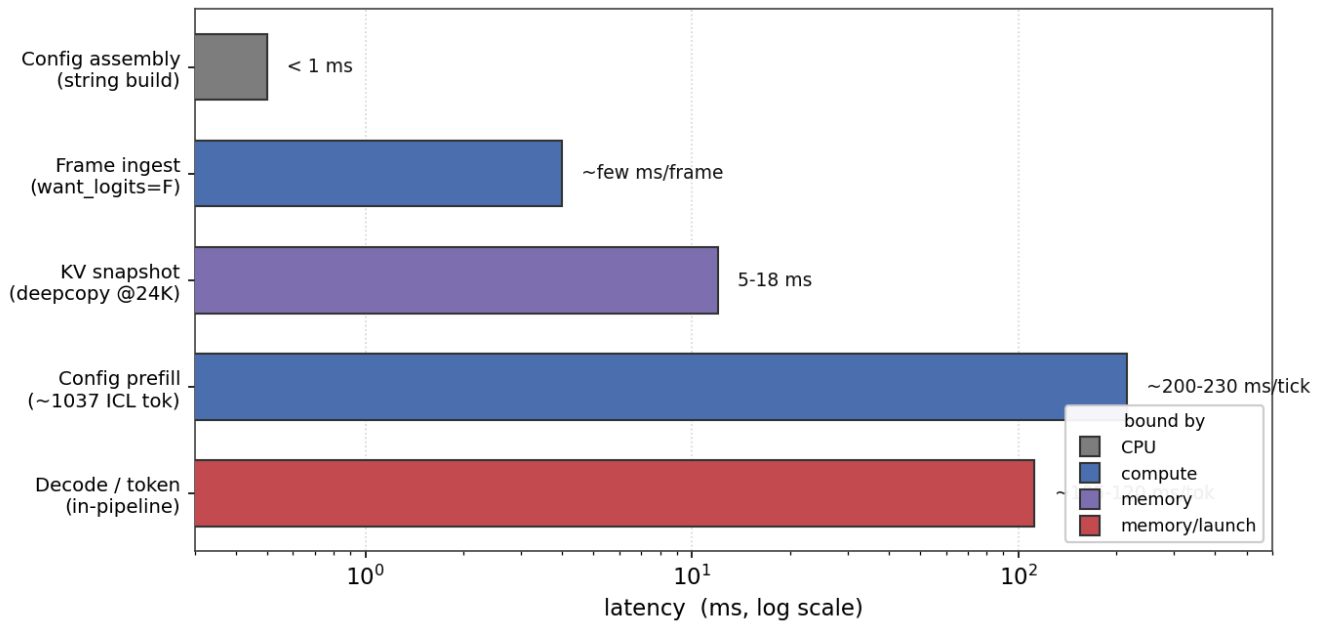


Figure 2. Per-stage latency (log scale). Decode per token dominates; config assembly is effectively free and config prefill is a modest ~0.2 s/tick.

Values are from the repository's profiler telemetry and the isolated latency study (GH200, 8B bf16, SDPA attention, single GPU). "Memory waste %" and coarse overhead bands are (*est.*) where the profiler does not directly measure them; they are stated as such.

Component / Pipeline Stage	Avg Latency	Memory Waste (%)	GPU Overhead	Description
Config assembly (prompt string build)	< 1 ms (CPU)	~0%	None	Python concat of ICL + history + cue (<code>controller.py:178</code>)
Config prefill (ICL prompt → cache)	~200–230 ms/tick	~5% (<i>est.</i> , <i>re-prefill of ~1,037 ICL tokens not reused</i>)	Compute-bound	<code>step(embed_text(prompt+"{ "))</code> on the clone
Prefill — frame ingest	~ few ms/frame (<code>want_logits=False</code>)	low	Compute-bound	ViT tokens appended; <code>lm_head</code> skipped (<code>manager.py:65</code>)
KV cache snapshot / clone	5–18 ms (even @24K)	~50% transient (<i>est.</i> : <i>full deepcopy = 2× cache momentarily</i>)	Memory-bound	<code>MVCC snapshot_clone deepcopy</code> (<code>manager.py:161</code>)
KV eviction (<code>torch.cat</code>)	tail-only (rare < 22% budget)	up to ~100% transient of evicted span (<i>est.</i>)	Memory-bound	<code>rebuild[sink recent]</code> (<code>manager.py:83</code>)

Async "state writer" (emit + history)	negligible (in-controller)	~0%	None (no I/O)	append to <code>reported[]</code> ; no disk/PCIe path (§3.4)
Decode (control JSON, streaming)	~105–120 ms/token (in-pipeline); ~45–94 ms/token isolated	—	Memory/launch-bound	autoregressive JSON gen (<code>controller.py:198</code>)

Reading the table. Decode is the dominant term and the only one worth optimising further; config *assembly* is free, config *prefill* is a modest ~0.2 s that could be cut with KV reuse of the static ICL block, and the "async writer" (in the controller path) is essentially free because there is no persistence I/O. In the probe-gate arm, the analogous "writer" cost is a real free-generation decode (~60 tokens × ~100 ms/token).

5. Dynamic Execution Example and System-Prompt Evolution

5.1 A complete SCA query, tick by tick

Task instruction (fills `{instruction}`): *"Alert whenever the video provides specific logistical details for the match, such as the date, location, or ticket pricing."* The seed system prompt therefore becomes:

```
You are a helpful assistant watching a live video stream. According to the video you
are watching, your task is: Alert whenever the video provides specific logistical
details for the match, such as the date, location, or ticket pricing.
```

This is prefilled once and pinned as the sink. Then, per tick (`next_check_vt` starts at `probe_min_s = 0.2`):

Tick @3.0 s — cache holds frames [0..3] + timestamps. Assembled prompt = SCA ICL (~1,037 tok) + `assistant: none` history + cue + `{`. Decoded diff:

```
{"seen":"fans buying green body paint","have_enough_info":false,"fps":1.0,
 "question_for_next":"Is the date, location or ticket price shown now?"}
```

Apply: `have_enough_info=false` → level stays low, no fire. `question_for_next` is stored;
`next_check_vt = 3.0 + next_check_s.`

Tick @6.0 s — the stored question is now prepended: *"You previously asked yourself: 'Is the date... shown now?'. Judge it now from the MOST RECENT frames."* Decoded diff omits unchanged fields:

```
{"seen":"crowd roaring in the stadium","have_enough_info":false}
```

Still low → no fire. (Transient fields `answer/seen/event_time_s` reset each tick; persistent `fps/`

next_check_s/question_for_next survive — controller.py:223-231.)

Tick @16.0 s — the poster with the date/venue appears. Decoded diff:

```
{"seen":"poster with match date and venue","have_enough_info":true,"event_time_s":16,
"answer":"The match date is August 14th and the location is Dairy Farmers Stadium.",
"question_for_next":"Is the ticket price shown now?"}
```

Apply: level goes **false**→**true** = *rising edge* → **FIRE**. `event_time_s=16` (clamped to `[vt-10, vt]`) is recorded as the trigger time; `reported[] now = [(16, "The match date...")]`. History passed to future ticks: `assistant @16s: The match date...`

Tick @17.0 s — same poster still on screen:

```
{"seen":"same date and venue poster","have_enough_info":true,"event_time_s":16,
"answer":"The match date is August 14th and the location is Dairy Farmers Stadium."}
```

Apply: level is true but `prev_level` is already true → **not** a rising edge; word-overlap with the last fired answer is ~1.0 (> 0.5) → **not distinct** → **no fire** (dedup by the semantic Schmitt gate, `controller.py:246-249`). This is how "keep reporting true" avoids double-alerting.

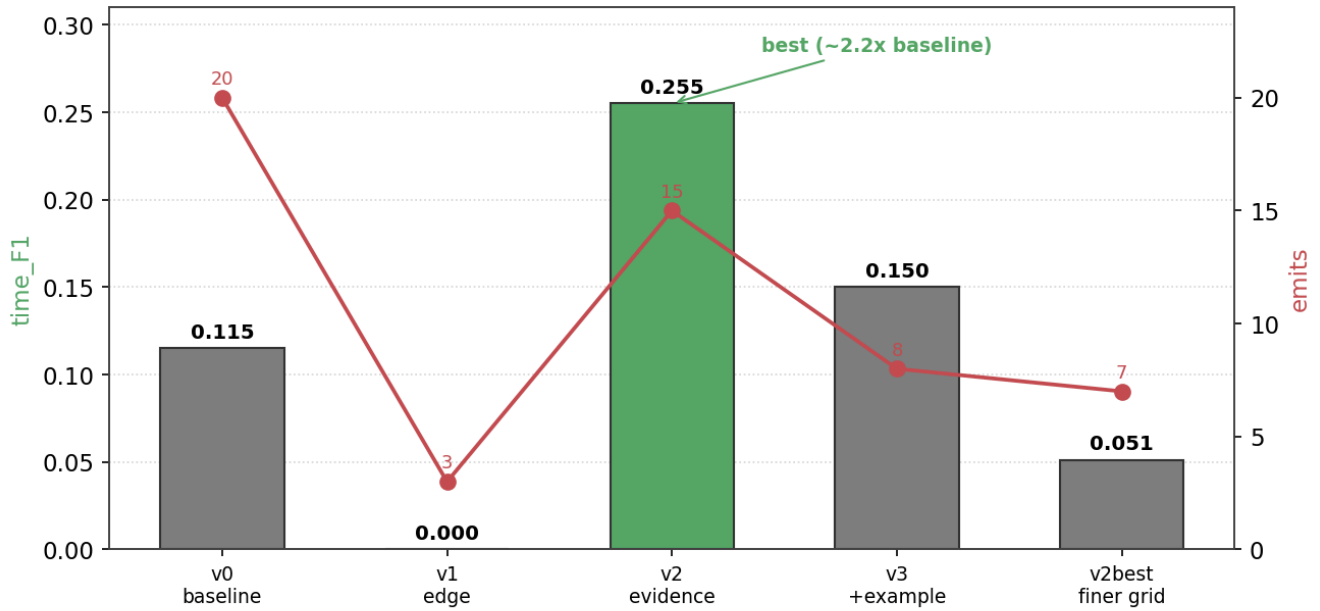
Tick @20.0 s — poster gone: `{"seen":"players celebrating","have_enough_info":false}` → level drops to false, `prev_level` reset.

Tick @23.0 s — ticket pricing appears; level rises again *with a clearly different answer* → **FIRE** (a new occurrence). `reported[]` now has two entries; `fps` bumped to 3 and `next_check_s` to 3 (sample sparsely, keep watching).

The user-visible output stream is exactly the two `answer` strings, timestamped at 16 s and 23 s — which is what the OmniPro scorer then matches against ground truth.

5.2 Prompt versioning and system evolution

Figure 5 Prompt-version lineage (SCA, frozen model) — F1 (bars) vs emit count (line)



Caveat (§6.2): these F1 deltas fall within run-to-run noise; the durable wins are edge-in-code + look-before-judge, not the exact ranking.

Figure 5. Prompt-version lineage on SCA (frozen model): time_F1 (bars) against emit count (line). v2-evidence is best; deltas are within run-to-run noise.

System_3 versions prompts *in code*, as named ICL variants; the evolution is documented in EXPERIMENTS.md. The lineage (all on the SCA task, frozen model, no training):

Version	Mechanism change	Emits	TP	time_F1	Verdict
v0 baseline	model self-dedups "in its head" (<code>new_event</code>)	20	3	0.115	re-fired same event 7×
v1 edge	model reports LEVEL; <code>code</code> fires on rising edge	3	0	0.000	too silent
v2 evidence	+ <code>seen</code> (look-before-judge) + <code>event_time_s</code>	15	5	0.255	best; ~2.2× baseline
v3	+ 2nd ICL example + calibration + lower gate	8	3	0.150	regressed (over-instructed)
v2best	v3-lineage + finer 0.2–1.5 s grid + <code>judge</code> ≥3	7	1	0.051	regressed further

Three durable design lessons emerged and are now baked into the default `controller_prompt` and the per-task ICL (`config.py`):

- 1. Move dedup from the model to the code.** A frozen 8B cannot reliably track "already reported"; letting it decide `new_event` caused 7× identical re-fires. The fix is *level*→*edge in code* (the model judges "is it true NOW"; the controller fires only on the rising edge). This is the single most important architectural version bump.
- 2. `seen` (look-before-judge) is the biggest accuracy lever.** Forcing the model to write what is on screen *before* the boolean is chain-of-thought compressed into one JSON field.

3. **More prompt is worse.** v3's extra example + calibration language *regressed* — a frozen 8B is easily over-instructed. The default therefore keeps exactly one worked example per task.

Caveat carried from the research log: the F1 deltas between these versions were later shown to be *within run-to-run noise* (§6.2). The qualitative lessons (edge-in-code, `seen`) held up; the exact F1 rank ordering did not. This is why the versioning discussion is framed as *mechanisms adopted*, not *scores achieved*.

6. Evaluation Frameworks (Online Mode & OmniPro)

6.1 Online-mode evaluation

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

Figure 6 Head-to-head (11 SCA videos) — tied on F1, opposite operating points

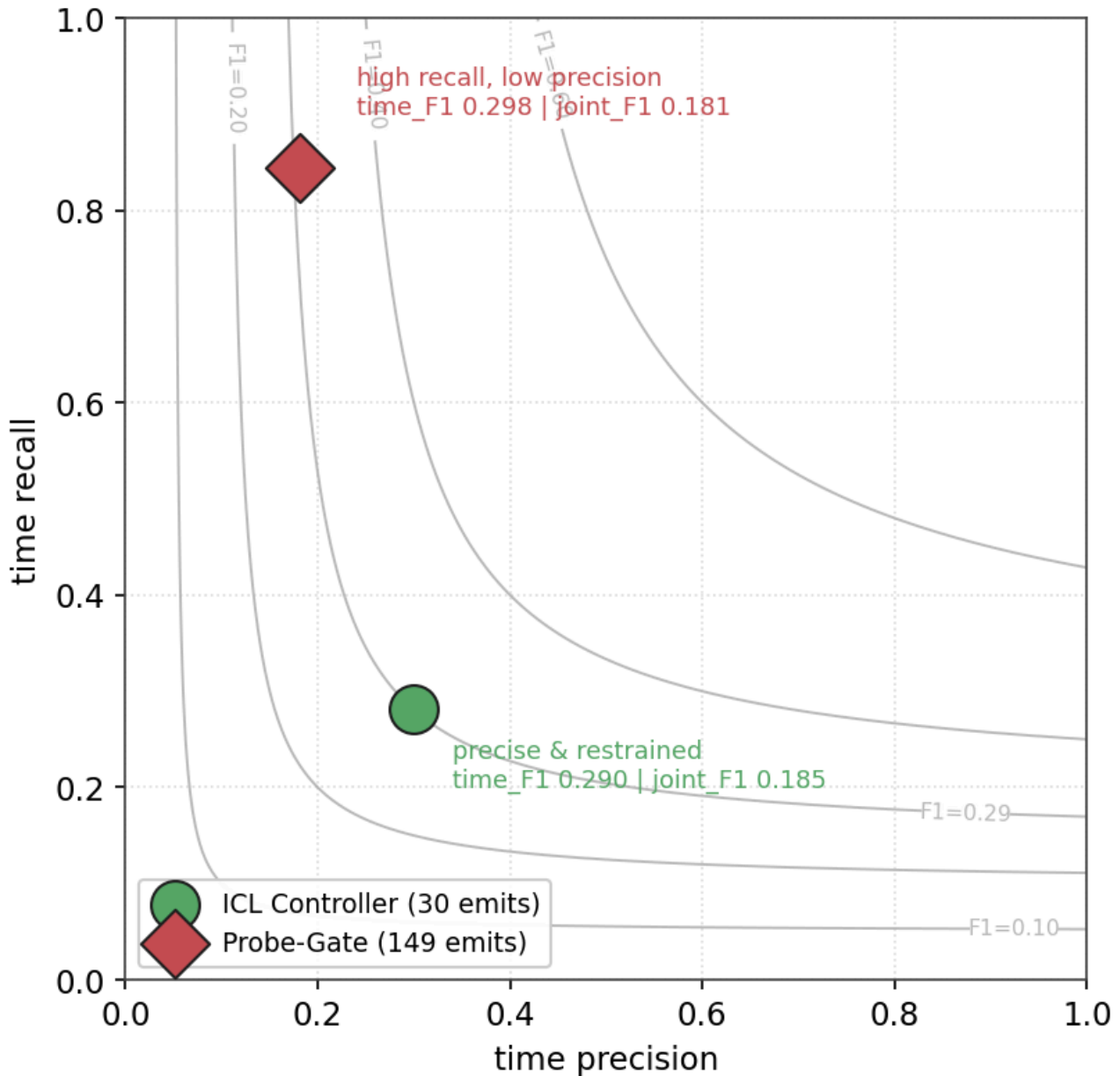


Figure 6. Controller vs. probe-gate on 11 SCA videos: statistically tied on F1 (iso-F1 contours) but at opposite precision/recall operating points.

`System_3` targets the **OmniPro online protocol** (arXiv 2605.18577). Its defining rule is that frames are fed **one by one at a fixed fps** (1 fps for online mode) and a response's timestamp is the *video-second at which the model emits it* — not wall-clock, not decode-finish time. The harness enforces this via the deterministic lockstep walk (§6.2), and the trigger timestamp is snapped to the video-time the decision *saw* (`vt` at loop-top), refined to the model-reported `event_time_s` clamped to `[vt-10, vt]` (`controller.py:239-267`).

Real-time monitoring signals. The profiler (`util.Profiler`) records the online-serving metrics per run and prints them into each sample's log (`system5_adapter.py:207`):

- **TTFT (time to first token)** $\approx$ per-tick prefill, ~ 0.2 s (`ctrl_prefill_s`).

- **TPOT (time per output token)** = `ctrl_decode_ms_per_tok`, ~105–120 ms/token in-pipeline.
- **Semantic-accuracy check** = the content-correctness gate applied only to temporally matched emissions (below).

Scoring (`metrics.py`). Emissions are matched to ground-truth triggers by **greedy, closest-first 1-to-1 temporal matching within ± 3 s** (`match_emits_to_gt`, `metrics.py:41–58`). From the match set:

$$P_{\text{time}} = \frac{tp}{tp + fp}, \quad R_{\text{time}} = \frac{tp}{tp + fn}, \quad F1_{\text{time}} = \frac{2P_{\text{time}}R_{\text{time}}}{P_{\text{time}} + R_{\text{time}}}.$$

Joint (content-gated) metrics count a matched emission only if it is *also* content-correct:

$$tp_{\text{content}} = \sum_{(e, g) \in \text{matches}} \mathbb{1}[\text{content_correct}(e, g)], \quad F1_{\text{joint}} = \frac{2P_{\text{joint}}R_{\text{joint}}}{P_{\text{joint}} + R_{\text{joint}}},$$

with $P_{\text{joint}}, R_{\text{joint}}$ formed from tp_{content} over the same fp, fn . Aggregation is **micro** (sum $tp/fp/fn$ across samples, then recompute). Content correctness is task-specific: exact-match on the extracted grid cell for ETG (`_extract_position`), integer match for counting, and **LLM-judge** for semantic-condition-alert / narration (`metrics.py:228–242`).

Measured head-to-head (11 SCA videos, audio=none, deterministic, seeded judge):

System	Emits	time_P	time_R	time_F1	joint_F1	content_acc
ICL Controller (<code>gate_mode=controller</code>)	30	0.300	0.281	0.290	0.185	0.556
Probe-Gate (<code>OMNIPRO_GATE_MODE=probe</code> , <code>hyst2b</code>)	149	0.181	0.844	0.298	0.181	0.519

The two are **statistically tied on F1 but occupy opposite operating points**: the controller is precise and restrained (30 emits); the probe is high-recall, low-precision (149 emits) — it re-fires every `gate_rearm_s=5 s` while a semantic condition stays true. Precision is the obvious lever for the probe arm (debounce / fire-only-on-newly-true).

6.2 The determinism problem and the lockstep fix

The most consequential eval finding is that **run-to-run variance initially exceeded every variant difference**. A near-identical config scored `time_f1` **0.255 vs 0.051** across runs.

Telemetry proved it was *not* frame drops (`frames_emitted == frames_ingested` on all 11 videos); it was an **async snapshot race**: the controller's snapshot each tick captured whatever the ingester had pushed *by that wall-clock instant*, and thread scheduling varied every run → different cache content per check → different greedy token → cascading divergence.

The fix (`cfg.deterministic=True`, default for eval) is a **frame-indexed lockstep walk**:

1. The encoder uses **fixed base fps** and a **blocking** `vis_q.put` (process every frame, no

drops) — `vision_stream.py:45,73`.

2. The ingester publishes `vt` **only after** a frame is fully in the cache, then **waits** until every controller tick due at $\leq vt$ has *completed* (`clock.get_next_check() > vt`) before feeding the next frame — `input_ingester.py:96-108`.
3. The controller publishes its `next_check` **only after** the fire+history update finishes, so the ingester's release is causally ordered (`controller.py:283-285`).

The result is that every tick provably sees exactly frames `[0..vt]`, identically on every run. This also matches the paper's "process frames one by one" semantics.

Residual non-determinism (honest caveat). Even with lockstep, two identical controller runs on the same clean GPU still diverged slightly (`time_f1` 0.1875 vs 0.1538 on a 3-video det check; `output_ctrl_detA` vs `output_ctrl_detB`). The cause is floating-point non-determinism in the autoregressive loop: `torch.use_deterministic_algorithms(True, warn_only=True)` **silently permits** non-deterministic ops (`util.py:44`), and a tiny logit difference eventually flips one `argmax` → a different control JSON → cascade. **Operational rule adopted:** run every eval $N \geq 3$ times and report the **mean** ($\pm$ spread), never a single run. Short clips reproduce exactly; longer clips accumulate divergence.

6.3 The OmniPro task suite and the judge

Task coverage. OmniPro defines nine online tasks (`dataset.ALL_TASKS`): instant / semantic event alerts, three counting variants, explicit target grounding, realtime state monitor, and two free-text narration tasks. `System_3`'s generative writer is natively suited to the *event/onset + free-text* subset (`SYSTEM5_NATIVE_TASKS`: instant/semantic alert + narration); counting/position/state need structured outputs and are handled per-task (grid-cell exact match for ETG). Only the `audio_dependency` $\in \{\text{none}, \text{helpful}\}$ subset is honestly attemptable because the pipeline **ingests no audio**; the headline subset is `none`, and the loader reports exactly what it kept rather than dropping silently (`dataset.py:8-13,133-136`).

Real-time simulation & state transitions. No *external environment* simulation is required — the "environment" is the pre-recorded video, replayed frame-by-frame. The only state transition computed online is the **semantic Schmitt gate**: the model reports the boolean `LEVEL` `have_enough_info` each tick, and the *code* computes the rising-edge / distinct-answer transition that constitutes a "fire" (§5.1). State (`reported[]`, `prev_level`, the persistent control `state` dict) transitions purely in the controller thread; there is no world-model rollout.

Judge LLM selection. Content for semantic-alert / narration is scored by **Google Gemini** (`ContentJudge`, `metrics.py:119-225`), default model `gemini-3.5-flash` (`GEMINI_MODEL`-overridable), via the `google-genai` SDK, with the REST OmniPro `llm_judge` and a lexical-overlap proxy as ordered fallbacks. It emits a **1–5 score**; the correctness threshold is ≥ 3 (the paper's protocol). Two properties make the judge reproducible — a hard requirement given §6.2:

1. The Gemini call is **seed-pinned** (`OMNIPRO_JUDGE_SEED`, default 1234) rather than relying on `temperature=0` (`metrics.py:201-208`).

2. Every verdict is **persisted** to `judge_cache.json`, keyed by `sha256(question, gt, pred)` (`metrics.py:179-189`). Re-scoring identical predictions returns the same joint-F1 at zero API cost.

Gemini (rather than GPT-4o / Claude) is used here because the surrounding OmniPro tooling and API base were Gemini-configured; the judge is provider-pluggable, so the choice is operational, not fundamental — the important properties are *seeded* and *cached*.

7. Critical Analysis: Failure Modes and Bottlenecks

7.1 Decode latency is the true bottleneck (not the cache, not config)

--	--	--	--	--	--

--	--	--

--	--	--

--	--	--	--

Figure 7 Decode latency — the true bottleneck (§7.1)

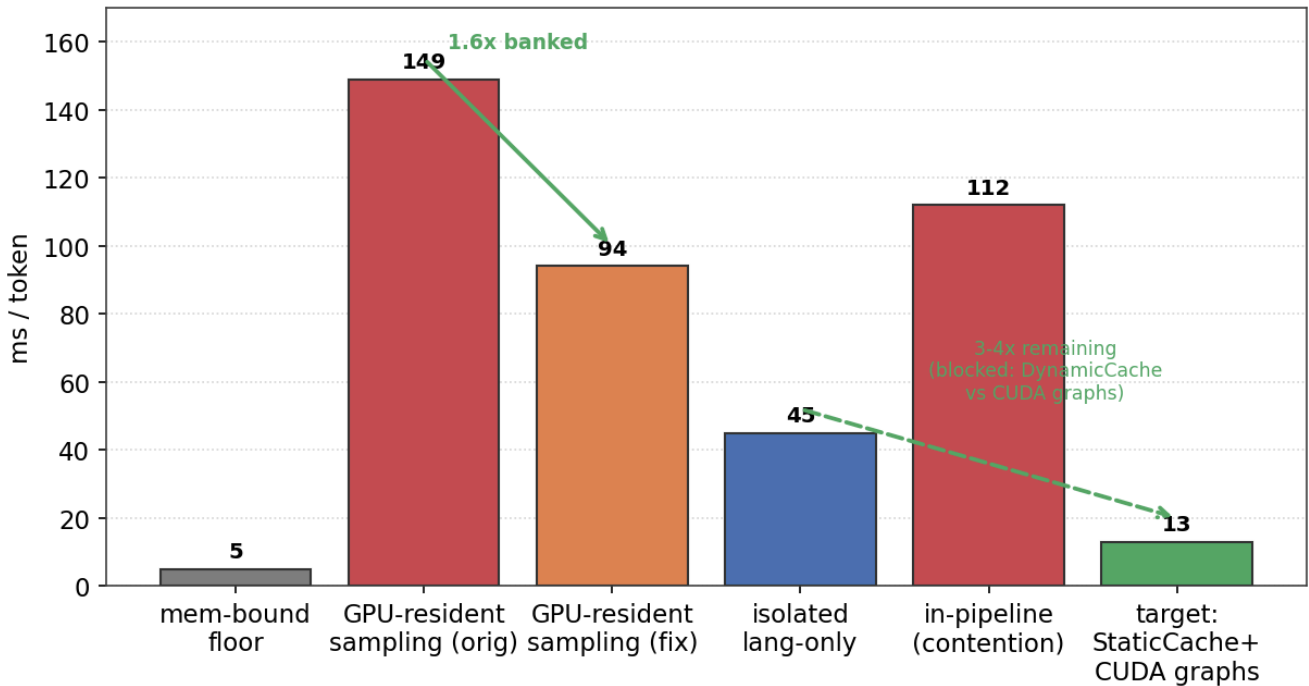


Figure 7. Decode latency: the memory-bound floor, the 1.6x sampling win already banked, in-pipeline contention, and the 3-4x StaticCache+CUDA-graph lever still blocked.

The isolated study (`qwen_only_latency_vs_icl_latency.md`) is unambiguous: raw language-only Qwen3-VL decodes at **~45 ms/token** on this stack — already ~9× above the ~5 ms memory-bound floor — because 36 layers of small kernels traverse the HF Python stack every token (per-call launch overhead, *flat across cache lengths 1k/8k/24k*, so it is **not** attention/memory-bound). In-pipeline the controller pays **~105–120 ms/token** (launch overhead + contention from the encoder/ingester sharing the GPU). Wins already banked: GPU-resident sampling (149→94 ms/tok, 1.6×), `want_logits=False` on ingest, and diff-decoding (quiet ticks ~14 tokens). The **only remaining large lever is StaticCache + CUDA graphs** (est. 45→~10–15 ms/tok, 3–4×) — but `torch.compile(reduce-overhead)` **fails on the growing DynamicCache** (CUDA graphs need a pre-allocated `StaticCache`), and flash-attention-2 is **not built for this aarch64 environment** (SDPA is the best available). Multi-GPU splitting did *not* help — the per-tick cross-GPU cache copy cancels the gain.

7.2 Fragmentation and reallocation in the single-tensor cache

Because the cache is one contiguous tensor per layer (not paged), two operations cause transient memory spikes and allocator churn:

- **Eviction** rebuilds each layer's K/V with `torch.cat([sink, recent])` (`manager.py:83-101`), which allocates a fresh tensor while the old one is still live — a transient ~2× of the kept span, and a source of allocator fragmentation on long streams. A paged/block cache would evict by dropping block references with zero copy; that is the standard fix and is not implemented here.
- **Snapshot** deep-copies the *entire* cache every tick (5–18 ms, cheap in time but 2× in

peak memory momentarily). At a 36 GiB full cache this would be a 36 GiB transient — fine on a 96 GB GH200, tight anywhere smaller. Copy-on-write or a shared read-only view would remove this, but conflicts with `DynamicCache`'s mutable layout.

Neither is a *current* failure on typical OmniPro clips (cache stays <22% of budget), but both are hard ceilings for truly long-horizon deployment.

7.3 Concurrency correctness and the two deadlocks that were fixed

The MVCC design is race-free *by construction* for cache coherence (single writer, snapshot readers), but the *thread lifecycle* produced two real deadlocks that are instructive:

- **Probe-gate writer deadlock.** The writer originally keyed its exit off the shared `stop` event, which the *encoder* sets at end-of-stream. It could exit while the ingester was still draining buffered frames; a late fire's `writer_q.put(); writer_q.join()` (the deterministic path) then blocked forever with no servicer — a stall *after* "video stream ended", GPU at 0% (a contention look-alike that is **not** contention). Fix: exit on `feed_done` (set by the ingester only after it finishes feeding), and move `writer_q.task_done()` into a `finally` so a generation exception can't strand the join (`writer.py:44-98`, `input_ingester.py:85-92`).
- **Controller drain.** Symmetrically, the controller must keep servicing ticks during the stream drain and only exit once `stop AND not due AND feed_done` (`controller.py:157`).

These are not races on the cache; they are producer/consumer lifecycle bugs, and they are the kind of failure that *looks* like GPU contention (see the shared-GPU caveat below) but is not.

7.4 Residual non-determinism (the eval's soft underbelly)

As quantified in §6.2, `warn_only=True` deterministic algorithms + a purely greedy but floating-point-sensitive argmax loop leave a small but real run-to-run variance that *compounds* on longer clips. The mitigation is procedural ($N \geq 3$ runs, report the mean), not algorithmic. A fully bit-reproducible loop would require `warn_only=False` (which currently throws on some kernels) or a tolerance-based decode. **Every single-run F1 in this repository is a point estimate with real variance and must be read as such.**

7.5 Config / prompt overhead: the re-prefill waste

The static ICL block (~1,037 tokens for SCA) is **re-embedded and re-prefilled every tick** because it is spliced onto a fresh clone each time (§4.2). This is ~0.2 s of avoidable compute per decision. Since the ICL text is *identical* across ticks and always sits at the same logical offset relative to the growing frame context, it is a natural candidate for KV-block reuse (prefill once, snapshot with the ICL already resident) — currently unexploited. This is the "token waste during dynamic prompt injection" the brief anticipates, and it is real but bounded (config prefill $\ll$ decode).

7.6 Model-capability ceiling and shared-hardware confounds

- **Recall ceiling.** Several SCA videos emit **zero** across *every* prompt variant — the frozen 8B genuinely never recognises those abstract/visual conditions. This is a comprehension limit, not a prompt bug, and it caps achievable recall. The proposed attack (compile the abstract condition into concrete visual sub-checks at t=0, VISPROG-style, then let `seen` verify them) is future work.
- **Shared-GPU contention.** The GH200 node is multi-user; a busy neighbour can push a tick from ~10 s to ~84 s (~1.5 s/token), which freezes the video clock and starves the ingester → **0 emits**. Per-token decode speed is independent of prompt wording, so slowness/0-emits after a prompt edit is *almost always* contention, not the change. The `ctrl.gate` log prints `gen=<s> ntok=<n>` precisely so this can be diagnosed; the rule is to check `nvidia-smi` and rerun on a genuinely free GPU before blaming code.

8. Summary

`System_3` is best understood not as an inference server but as a **cognitive architecture over a single frozen VLM**: one shared linear KV cache, a single-writer ingester, and a generative controller that emits an in-context-taught JSON control language which *is* the perception scheduler, the output gate, and the writer at once. Its genuine engineering contributions are the **MVCC snapshot** decoupling of readers from the sole writer, the **dual logical/physical clocks** that let eviction and RoPE coexist, the **GPU-resident decode path** (a real 1.6×), and the **deterministic lockstep walk + seeded/cached Gemini judge** that finally made the eval comparable. Its honest limitations are equally clear: the cache is contiguous (not paged), there is **no** asynchronous KV persistence backend (the "async" is thread-level MVCC, not GPU→CPU/disk offload), decode latency dominates and is blocked from its last 3–4× by the DynamicCache/CUDA-graph incompatibility, a residual floating-point non-determinism forces averaged reporting, and the frozen 8B imposes a recall ceiling no prompt can lift. The head-to-head verdict — controller and probe **tied on F1 at opposite precision/recall operating points** — is the current research frontier, to be resolved (per `EXPERIENCE.md`) by locking the per-task winner and expanding task-by-task in mechanism-reuse order.

Appendix A — Key file:line index

Concept	Location
Static config dataclass	<code>async_omni_v2/config.py:205</code>
Control-JSON DSL (generic)	<code>async_omni_v2/config.py:270–291</code>
Per-task ICL prompts	<code>async_omni_v2/config.py:38–202</code> , <code>registered :341–345</code>
Shared cache + MVCC snapshot	<code>async_omni_v2/manager.py:161–193</code>
Eviction (StreamingLLM, sink)	<code>async_omni_v2/manager.py:83–101</code>

GPU-resident forward / want_logits	async_omni_v2/backend.py:213-242
Controller loop (prefill/decode/fire)	async_omni_v2/controller.py:150-285
Level→edge semantic Schmitt gate	async_omni_v2/controller.py:246-249
Deterministic lockstep (ingester)	async_omni_v2/input_ingester.py:96-108
Temporal match + F1	omniprofast/metrics.py:41-58, 273-304
Gemini judge (seeded + cached)	omniprofast/metrics.py:119-225
Fidelity contract (eval spins real pipeline)	omniprofast/system5_adapter.py:1-18, 115-201

Appendix B — Reproduction

```
# ICL controller (default) on the 11-video SCA subset, audio=none
omniprofast/run_fast.sh --tasks semantic_condition_alert --audio none \
  --benchmark_json <...>/omnipro_data/benchmark.json --out ./output_ctrl

# Probe-gate arm from the same checkout (A/B via env var)
OMNIPRO_GATE_MODE=probe omniprofast/run_fast.sh --tasks semantic_condition_alert \
  --audio none --benchmark_json <...>/benchmark.json --out ./output_probe

# NOTE: run N>=3 times and report the MEAN (eval is not bit-reproducible; $6.2).
```

Backbone assumed for the memory model: Qwen3-VL-8B-Instruct text decoder
L = 36 layers, H_q = 32 query heads, H_kv = 8 KV heads (GQA), D = 128, bf16.
Per-token KV = 2·36·8·128·2 = 147,456 B ≈ 144 KiB; @262,144 tok ≈ 36 GiB.